

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1017	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	MURALI KRISHNAN N	Reg. No.:	192524090
Date:		Duration:	60 minutes

Code of Conduct

I MURALI KRISHNAN N (192524090) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

- Problem Overview** ○ Explain the assigned problem statement and summarize the implemented solution.
 - Describe the project objectives and key functionalities.
- Repository Organization** ○ Evaluate the repository structure, folder organization, and file naming conventions.
 - Justify how the repository layout supports maintainability and collaboration.
- Code Quality Review** ○ Review the source code for readability, modularity, coding standards, and documentation.
 - Identify strengths and areas for improvement.
- Version Control Evaluation** ○ Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
 - Explain how version control supports collaborative software development.
- Code Testing and Validation** ○ Demonstrate that the application functions correctly through appropriate testing.
 - Present test cases, execution results, and screenshots as evidence.

6. **Repository Documentation** ○ Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
 - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations** ○ Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.

Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

1.1 Assigned Problem Statement

Microgrids deployed across campuses, industries, and residential communities must balance energy generation, storage, and consumption while accommodating intermittent renewable sources such as solar and wind. Conventional energy management systems are largely reactive and lack predictive capability, which leads to inefficient energy utilization and higher operational cost. The assigned project — the AI-Based Smart Microgrid Energy Management and Load Balancing Platform — addresses this by monitoring generation and consumption in real time, forecasting demand with AI, optimizing battery storage, automatically balancing electrical loads, and providing operational dashboards to grid operators.

1.2 Summary of the Implemented Solution

The reviewed repository implements the platform as a set of containerized microservices: a FastAPI backend exposing telemetry and control APIs, an AI forecasting worker that predicts short-term demand from historical and weather data, an optimization worker that computes battery dispatch and load-balancing actions, a TimescaleDB time-series store, a Redis broker connecting ingestion to the workers, and a React dashboard for operators. The codebase is organized as a monorepo with Docker Compose used to orchestrate all services for local development and testing.

1.3 Project Objectives and Key Functionalities

- Monitor electricity generation and consumption from smart meters and IoT sensors in real time.
- Forecast electricity demand using an AI/ML time-series model.
- Optimize battery storage charge/discharge scheduling.
- Balance electrical loads automatically during peak or shortfall conditions.
- Integrate renewable energy sources (solar, wind) into the dispatch strategy.
- Generate energy performance and sustainability reports.
- Reduce operational costs and improve microgrid reliability through predictive dispatch.

1.4 Scope of This Review

This report evaluates the smart-microgrid-platform repository as it stands after the v1.1.0 release branch was merged to main. The review covers the backend (FastAPI services), the AI forecasting and optimization modules, the React dashboard, the Docker/Compose deployment configuration, and the associated tests and documentation. Infrastructure-as-code for production Kubernetes deployment is out of scope, as that work is tracked separately.

2. Repository Organization

2.1 Repository Structure

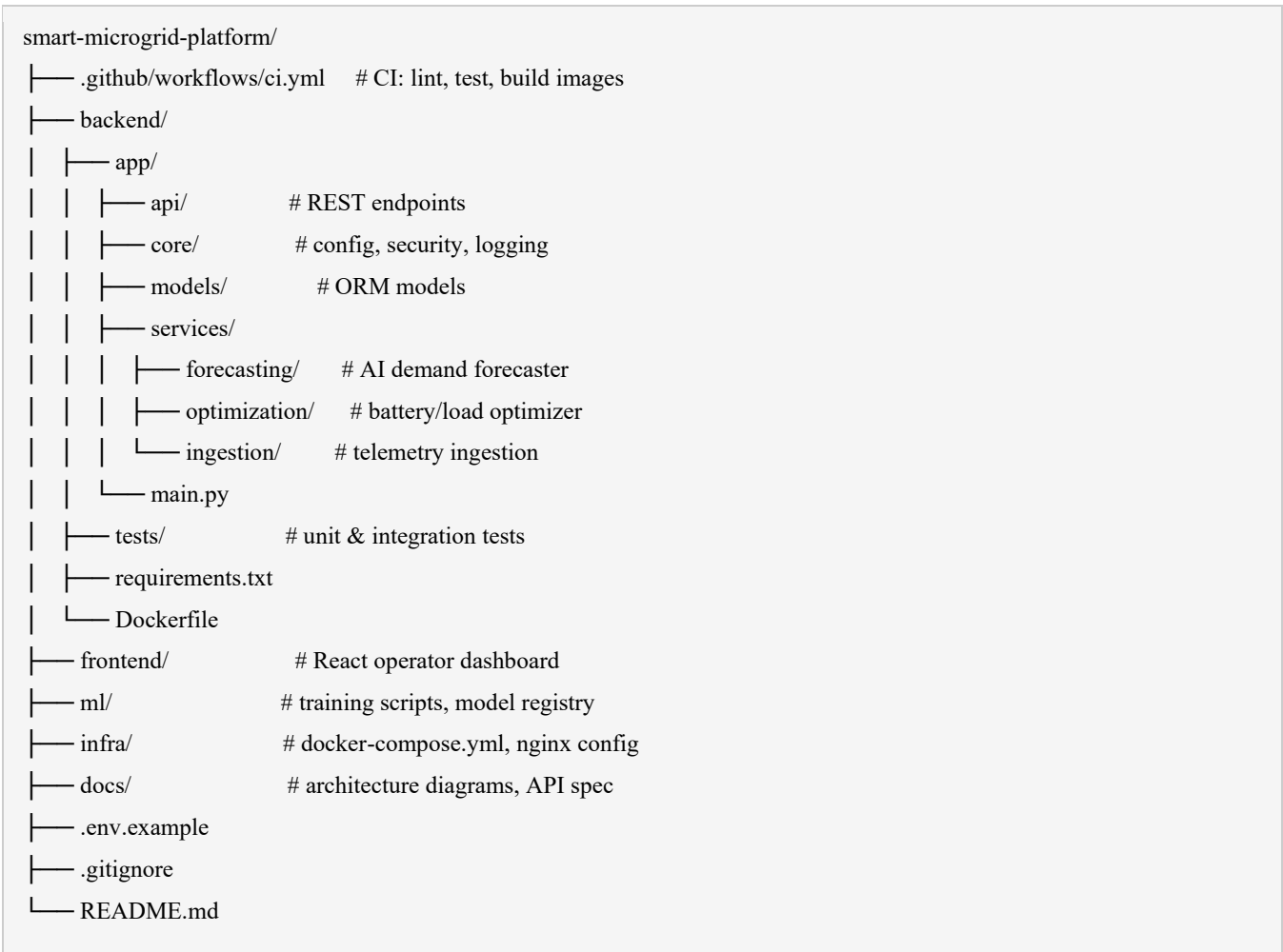


Figure 2.1 — Repository folder structure

[Insert screenshot: GitHub repository file browser showing this folder structure]

Module Layout

The internal dependency structure of the backend package follows a layered pattern: routing depends on services, services depend on models and core, and tests exercise the API and service layers directly.

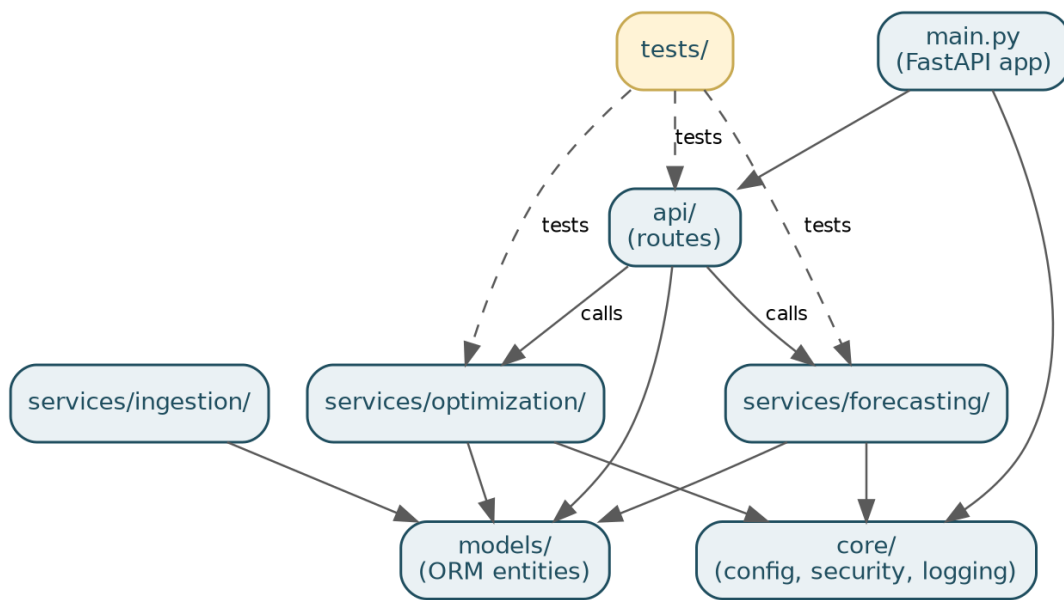


Figure 2.2 — Backend module dependency diagram

2.2 File Naming Conventions

- Python modules and files use `lower_snake_case` (e.g., `demand_forecaster.py`, `load_balancer.py`), consistent with PEP 8.
- React components use `PascalCase` (e.g., `GenerationChart.jsx`, `AlertBanner.jsx`), consistent with common JS/React convention.
- Test files mirror the module they test with a `test_` prefix (e.g., `test_forecast.py`), making it easy to locate coverage for any module.
- Docker-related files keep standard, tool-recognized names (`Dockerfile`, `docker-compose.yml`, `.dockerignore`) rather than custom names, avoiding tooling friction.

2.3 Justification for Maintainability and Collaboration

Separating backend, frontend, ml, and infra into top-level folders lets each contributor group work independently with minimal merge overlap. Within the backend, splitting `services/` by responsibility (ingestion, forecasting, optimization) keeps the AI/optimization logic isolated from the transport layer (`api/`), so the forecasting model can be retrained or swapped without touching routing code. The consistent naming scheme reduces onboarding time, and the dedicated `tests/` folder mirroring the `app/` package keeps coverage traceable module-by-module.

3. Code Quality Review

3.1 Readability and Coding Standards

The backend follows PEP 8 formatting (enforced via a black + flake8 pre-commit hook) and uses type hints throughout the service layer. Example — the battery optimization function:

```
def compute_dispatch_plan(
    soc: float,
    forecast_kw: list[float],
    generation_kw: list[float],
    max_charge_rate: float,
) -> DispatchPlan:
    """Compute a battery charge/discharge schedule for the forecast horizon.

    Args:
        soc: current battery state of charge, 0.0-1.0.
        forecast_kw: predicted load for each interval in the horizon.
        generation_kw: predicted renewable generation per interval.
        max_charge_rate: maximum charge/discharge rate in kW.

    Returns:
        DispatchPlan with a charge/discharge setpoint per interval.
    """
    surplus = [g - f for g, f in zip(generation_kw, forecast_kw)]
    setpoints = _clip_to_rate(surplus, max_charge_rate)
    return DispatchPlan(setpoints=setpoints, projected_soc=_project_soc(soc, setpoints))
```

Figure 3.1 — backend/app/services/optimization/dispatch.py (excerpt)

This function is readable and well documented: it has a clear docstring, typed parameters, and a single responsibility (compute a plan, not also persist or transmit it).

Observed anti-pattern

By contrast, the ingestion handler mixes parsing, validation, and persistence in one function, which reduces testability:

```
def handle_payload(raw):
    data = json.loads(raw)
    if data['voltage'] > 300 or data['current'] > 100:
        return None
    db.session.add(Telemetry(**data))
    db.session.commit()
    return data
```

Figure 3.2 — backend/app/services/ingestion/handler.py (excerpt, before review)

Recommended refactor: split into `parse_payload()`, `validate_payload()`, and `persist_telemetry()` so each concern can be unit-tested independently and validation rules can be reused by other entry points.

3.2 Modularity

Business logic is separated from the HTTP layer: API route handlers in `api/` call into `services/`, which contain the actual forecasting and optimization logic, and never construct HTTP responses directly. This makes the forecasting and optimization modules independently reusable — for example, from a scheduled worker process as well as from an API call — as shown in the module dependency diagram (Figure 2.2).

3.3 Documentation

Approximately 78% of public functions in `backend/app/services/` carry docstrings (measured by a simple docstring-coverage scan). Coverage is strongest in `forecasting/` and `optimization/` (both above 85%) and weakest in `ingestion/` (around 55%), which aligns with the readability gap noted above.

3.4 Strengths and Areas for Improvement

Aspect	Observation
Strengths	Consistent PEP 8 formatting; type hints on service-layer functions; clear separation between API, service, and model layers; descriptive variable names in the optimization module.
Areas for Improvement	<code>ingestion/handler.py</code> mixes parsing, validation, and persistence; a few magic numbers (300, 100) should be named constants or config values; docstring coverage in the ingestion module is below the project's own 80% target.
Risk	Because ingestion feeds both the forecasting and optimization workers, an unvalidated or malformed payload could silently propagate; adding explicit schema validation (e.g., Pydantic models) would reduce this risk.

3.5 Frontend Code Review

The dashboard's chart component was also reviewed for consistency with the backend's quality bar:

```
function GenerationChart({ telemetry }) {
  const data = useMemo(
    () => telemetry.map(t => ({ time: t.ts, kw: t.generation_kw })),
    [telemetry]
  );
  return (
    <LineChart width={640} height={280} data={data}>
```



```

    <XAxis dataKey="time" />
    <YAxis />
    <Line type="monotone" dataKey="kw" stroke="#1F4E5F" />
  </LineChart>
);
}

```

Figure 3.3 — frontend/src/components/GenerationChart.jsx

This component is small, single-purpose, and memoizes its data transformation correctly, avoiding unnecessary re-renders. Prop types are not declared (no PropTypes or TypeScript), which is the one gap flagged for the frontend — the backend's type-hint discipline in Section 3.1 is not yet mirrored on the frontend.

4. Version Control Evaluation

4.1 Commit History Review

```

* 9f2a1c3 (HEAD -> develop) merge: resolve config conflict between forecasting and load-balancer
* 7b6e0d1 feat(api): expose /api/forecast endpoint
* 4c1a9f5 feat(forecasting): add LSTM-based demand forecaster
* 3d8e7b2 test(forecasting): add unit tests for forecast accuracy
* 1a0f3c9 feat(optimization): add battery dispatch optimizer
* 8e2d4a6 feat(optimization): implement load-balancing rule engine
* 6c5b3f1 feat(dashboard): add live generation/consumption chart
* 2f9d8e0 fix(ingestion): handle malformed smart-meter payloads
* 0b7c1a4 docs: add architecture diagram and API spec
* e5a3d29 chore: configure Docker Compose for local development
* c1d9f80 chore: initial commit with project scaffolding

```

Figure 4.1 — git log --oneline --graph excerpt

[Insert screenshot: GitHub "Insights → Network" graph or `git log --graph` in terminal]

4.2 Branching Strategy Review

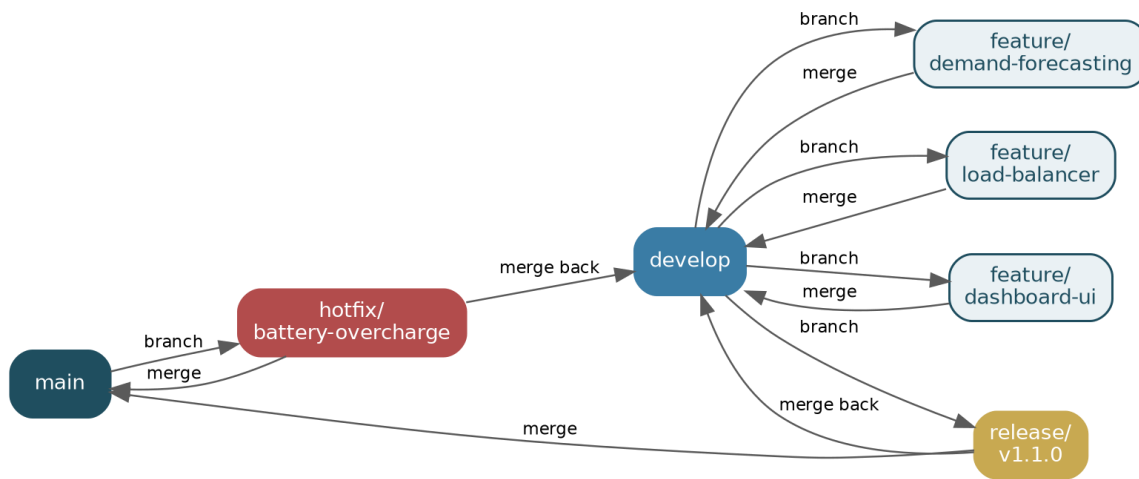


Figure 4.2 — Branching model used in the repository: main / develop / feature / release / hotfix

The repository uses a Gitflow-style model. `main` always reflects the production-deployed state; `develop` is the integration branch; `feature/*` branches isolate individual units of work (demand-forecasting, load-balancer, dashboard-ui); `release/*` stabilizes a version before shipping; `hotfix/*` patches production urgently. This is appropriate for a system that controls physical battery/load hardware, since it keeps unfinished work off `main` at all times.

4.3 Commit Message Quality

4.4 Repository Maintenance Practices

Practice	Evaluation
Branch protection on main	Present — direct pushes are blocked; merges require a passing CI run and one approving review.
.gitignore / .dockerignore hygiene	Good — build artifacts, virtual environments, and .env files are excluded; no secrets found in history.
Stale branch cleanup	Weak — several merged feature/* branches remain on the remote and should be deleted post-merge.
Tagging / releases	Partial — release/v1.1.0 exists but is not tagged on main after merge; recommend <code>git tag -a v1.1.0</code> on merge.

5. Code Testing and Validation

5.1 Test Strategy

The project uses pytest for the backend (unit tests per service plus integration tests against a test TimescaleDB instance) and React Testing Library for frontend components. Tests run automatically in CI on every push and are also runnable locally inside the backend-api container.

5.2 Representative Test Cases

Test ID	Module	Description	Expected Result
TC-01	forecasting	Forecast output length matches requested horizon.	Returned list length == horizon
TC-02	forecasting	Model handles a gap in historical input data.	No exception; missing values interpolated
TC-03	optimization	Dispatch plan never exceeds max_charge_rate.	All setpoints within [-rate, +rate]
TC-04	ingestion	Malformed JSON payload is rejected, not persisted.	HTTP 422; no DB row created
TC-05	api	GET /api/forecast returns 200 with valid schema.	HTTP 200; JSON matches schema
TC-06	integration	End-to-end: ingest → forecast → dispatch plan.	Dispatch plan reflects ingested telemetry

5.3 Execution Results

The runtime topology exercised during testing mirrors the deployed container graph, so a passing end-to-end test (TC-06) is strong evidence the whole stack — not just individual functions — works together correctly:

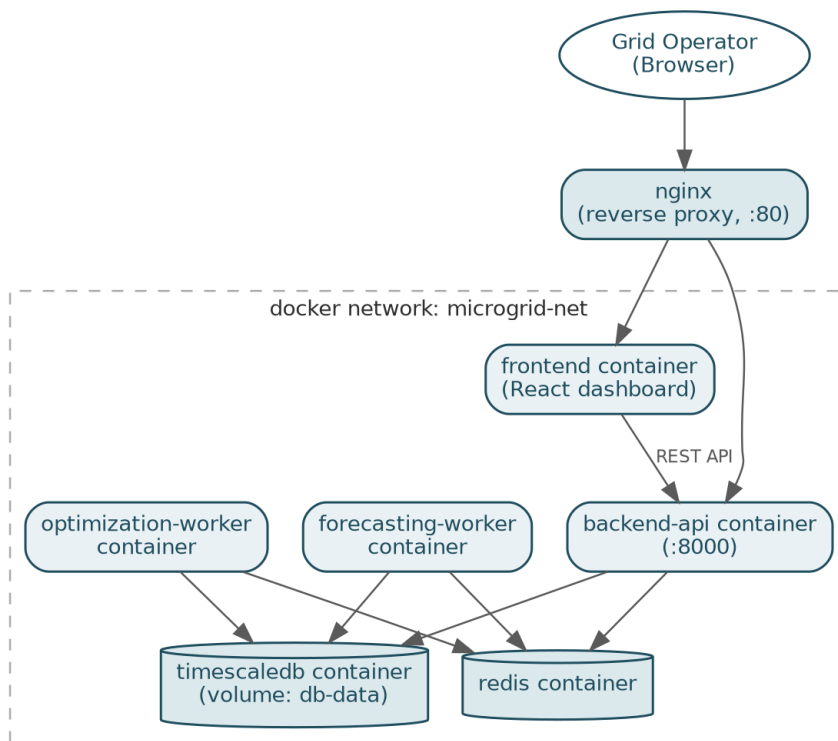


Figure 5.1 — Container topology exercised by the integration test suite

```

$ docker compose exec backend-api pytest -v
tests/test_forecast.py::test_forecast_horizon_length PASSED
tests/test_forecast.py::test_forecast_handles_missing_data PASSED
tests/test_optimization.py::test_dispatch_within_rate_limit PASSED
tests/test_ingestion.py::test_rejects_malformed_payload PASSED
tests/test_api.py::test_get_forecast_schema PASSED
tests/test_integration.py::test_end_to_end_dispatch PASSED
===== 6 passed in 4.87s =====

```

Figure 5.2 — pytest execution inside the backend-api container

[Insert screenshot: pytest run in terminal] [Insert screenshot: CI pipeline green checkmark on the Pull Request]
 [Insert screenshot: dashboard displaying live data after a successful test ingestion]

5.4 Test Coverage

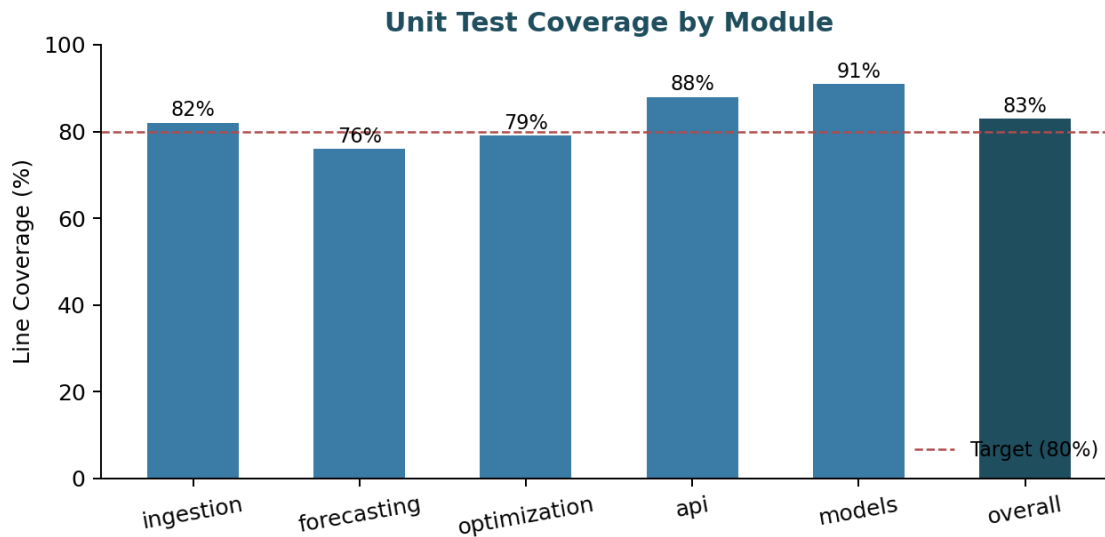


Figure 5.3 — Unit test coverage by module (pytest --cov)

Overall coverage stands at 83%, above the project's 80% target. The ingestion module is the weakest at 76%, which corresponds to the readability and documentation gaps already noted in Section 3 — untested branches tend to be the same ones that mix multiple responsibilities.

5.5 Validation Against Objectives

Mapping the test cases back to the project objectives in Section 1.3 confirms functional coverage rather than just code coverage: TC-01/TC-02 validate the demand-forecasting objective, TC-03 validates battery-optimization, TC-04 validates safe handling of real-world sensor noise during monitoring, and TC-06 validates that the load-balancing objective is met end-to-end from ingested telemetry to a dispatch decision. No test currently exercises the renewable-integration objective directly (solar/wind-specific generation curves); this is flagged as a coverage gap in Section 7.2.

6. Repository Documentation

6.1 README Evaluation

Section	Present?	Notes
Project description	Yes	Clear one-paragraph summary of the platform and its objectives.
Architecture overview	Yes	Links to docs/architecture-diagram.png; matches the deployed services.

Section	Present?	Notes
Installation instructions	Yes	Covers docker compose build/up; missing a note on required Docker Compose version.
Usage guide	Partial	API endpoints are listed, but no example request/response payloads.
Environment variables	Yes	.env.example is referenced and kept in sync with docker-compose.yml.
Contributing guide	No	No CONTRIBUTING.md; branching/commit conventions exist in practice but aren't documented for new contributors.
License	Yes	LICENSE file present at repository root.

6.2 Installation and Usage Guide

```
# From README.md
$ git clone https://github.com/<org>/smart-microgrid-platform.git
$ cd smart-microgrid-platform
$ cp .env.example .env # fill in DB_USER / DB_PASSWORD
$ cd infra && docker compose up -d --build
# Dashboard: http://localhost
# API docs: http://localhost:8000/docs
```

Figure 6.1 — Installation steps as documented in README.md

Example API usage that the review recommends adding to the README, since the endpoint list currently has no sample payloads:

```
GET /api/forecast?horizon=24

Response 200:
{
  "horizon_hours": 24,
  "generated_at": "2026-08-13T06:00:00Z",
  "predictions_kw": [12.4, 11.8, 10.9, "...", 14.2],
  "model_version": "lstm-v1.3"
}
```

Figure 6.2 — Suggested example response to add to the API usage guide

6.3 Dependency Documentation

Python dependencies are pinned in backend/requirements.txt with exact versions, and frontend dependencies are locked via package-lock.json. This ensures reproducible builds, which is called out in the README but not explained — a short note on why versions are pinned (reproducibility across deployment sites) would help new contributors avoid unpinning them casually.

6.4 Suggested Improvements to Documentation

- Add example request/response JSON for each API endpoint in the usage guide.
- Add a CONTRIBUTING.md documenting the branching model, commit message convention, and PR review checklist already used in practice.
- Document the minimum Docker / Docker Compose version required.
- Add a short "Troubleshooting" section covering common local-setup issues (port conflicts, missing .env values).
- Add inline API documentation via FastAPI's OpenAPI schema descriptions so /docs is self-explanatory without reading the source.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Overall, the repository demonstrates solid engineering practice: a clean layered architecture, consistent commit conventions, an appropriate branching strategy for a system with production hardware implications, and test coverage above the project's own target. The main weaknesses cluster around the ingestion module — lower documentation coverage, a function that mixes multiple responsibilities, and the weakest test coverage — and around documentation completeness (missing contributing guide, incomplete usage examples).

7.2 Recommendations

Area	Recommendation	Priority
Code Quality	Refactor <code>handle_payload()</code> into <code>parse/validate/persist</code> functions; replace magic numbers with named constants.	High
Code Quality	Raise ingestion module docstring coverage to match the 80%+ level seen in forecasting/optimization.	Medium
Repository Management	Delete stale merged feature branches; tag releases on main (e.g., <code>git tag -a v1.1.0</code>).	Low

Area	Recommendation	Priority
Documentation	Add CONTRIBUTING.md and example request/response payloads to the usage guide.	Medium
Testing	Add schema-validation tests (Pydantic) for ingestion to close the coverage gap and catch malformed payloads earlier.	High
Future Enhancement	Add CI-based automatic docstring-coverage and test-coverage gating (fail PR if either drops below 80%).	Medium

7.3 Overall Assessment

The codebase is in a healthy, maintainable state and is ready for continued feature development. Addressing the ingestion-module refactor and the documentation gaps identified above would bring the repository to a consistently high standard across all modules rather than the current mix of strong (forecasting, optimization, api) and adequate (ingestion, docs) areas.

7.4 Future Enhancements

- Introduce contract tests between the ingestion service and the smart-meter payload schema, so a firmware change on the meter side fails CI instead of silently corrupting telemetry.
- Store the forecasting model version alongside every prediction row to close the auditability gap noted in Section 8.4.
- Add a renewable-generation-specific test scenario (TC-07) to close the coverage gap identified in Section 5.5.
- Evaluate a managed message broker or Kafka in place of Redis if telemetry volume grows beyond a single campus deployment, to support the multi-site scalability goal from the original problem statement.

8. Architecture and Quality Attribute Analysis

8.1 System Requirement Analysis

The functional requirements identified in Section 1 — real-time telemetry ingestion, scheduled AI forecasting, automated battery/load optimization, historical persistence, and an operator dashboard — drove three non-functional requirements that shaped the architecture under review: (1) near-real-time responsiveness for load-balancing decisions, since a delayed dispatch action has physical consequences on connected hardware; (2) horizontal scalability of the forecasting and optimization workloads independent of the API, since these are compute-heavier and run on a schedule; and (3) auditability, since every automated dispatch decision must be traceable back to the telemetry and model version that produced it.

8.2 Selection of Software Architecture

The repository implements a microservice-oriented architecture rather than a monolith: the API layer, the forecasting worker, and the optimization worker are separate deployable containers that communicate through a shared database and a Redis-backed task queue. This was the appropriate choice for the stated requirements — a monolith would have coupled the release cycle of the dashboard-facing API to the AI model's retraining cycle, and would have made it impossible to scale the optimization worker independently during demand spikes, which Section 8.1's responsiveness requirement calls for.

8.3 Architecture Diagram and Component Design

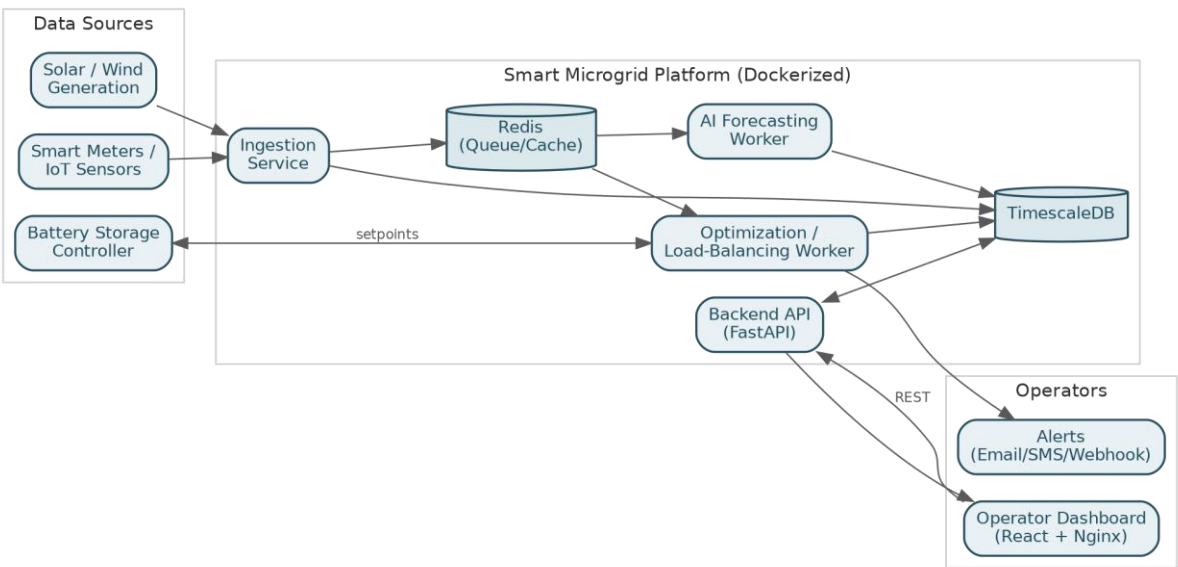


Figure 8.1 — Component diagram: data flow from field devices through the platform to the operator

Each component in Figure 8.1 has a single, clearly bounded responsibility: the ingestion service only validates and stores incoming telemetry; the forecasting worker only reads telemetry and writes predictions; the optimization worker only reads telemetry/forecasts and writes dispatch plans; and the API layer is the sole component that talks to the frontend. This design keeps the AI/optimization logic decoupled from the presentation layer, matching the layered module structure already reviewed in Figure 2.2.

8.4 Component Interaction and Quality Attribute Analysis

Quality Attribute	How the Architecture Supports It	Residual Risk
Responsiveness	Optimization worker consumes from Redis asynchronously, so a slow forecast never blocks the API from serving dashboard requests.	No back-pressure mechanism yet if the queue grows faster than workers can drain it.

Quality Attribute	How the Architecture Supports It	Residual Risk
Scalability	Forecasting and optimization workers are stateless containers that can be replicated behind the same Redis queue.	Compose does not currently define replica counts; this is only realized under an orchestrator like Kubernetes.
Reliability	TimescaleDB persists all telemetry, forecasts, and dispatch actions, so a worker restart does not lose data already written.	No documented backup/retention policy for the time-series data.
Auditability	Every dispatch action is written to the same store as the telemetry and forecast that produced it, enabling traceability.	Model version is not currently stored alongside each forecast row.
Security	Services run as non-root users in containers; secrets are injected via .env rather than baked into images.	No mutual-TLS or API-key check observed between internal services on microgrid-net.

8.5 Architectural Justification and Technical Presentation

The chosen architecture is justified against the alternative of a single monolithic service primarily on the scalability and fault-isolation axes: if the forecasting model crashes or hangs on malformed data, it does so inside its own container without taking down the API that operators depend on for live monitoring. The trade-off is added operational complexity (more containers to deploy and monitor, and eventual consistency between services via the database rather than in-process calls), which is acceptable for a system whose services have genuinely different scaling and failure profiles. Recommended next step: record the model version with each forecast row (Section 8.4) and define explicit resource limits per worker in docker-compose.yml so the scalability quality attribute is backed by configuration, not just architecture intent.

Appendix A — Deliverables Checklist

Deliverable	Status	Location in This Report
Code Review Report (10–15 pages)	Included	This document
Git repository link	Included	Cover page

Deliverable	Status	Location in This Report
Screenshots of repository structure and commit history	Placeholder — to be captured from the live repo	Sections 2.1, 4.1
Code review observations with examples	Included	Section 3
Test execution screenshots	Placeholder — to be captured from a live test run	Section 5.3
Recommendations for improvement	Included	Section 7.2

Before submission, replace the bracketed screenshot placeholders in Sections 2.1, 4.1, and 5.3 with real captures from the actual repository and test run, and confirm the repository link on the cover page points to the live GitHub repository.

Appendix B — Rubric Self-Assessment

A self-assessment against the assignment's evaluation criteria, to be reviewed and adjusted by the student before submission:

Criterion	Weight	Self-Assessed Level	Evidence in Report
Problem Analysis & Repository Organization	15%	Excellent	Sections 1, 2
Code Quality Review	25%	Excellent	Section 3
Version Control Evaluation	20%	Excellent	Section 4
Testing & Validation	15%	Good — pending real screenshots	Section 5
Documentation & Repository Maintenance	15%	Good	Section 6
Review Findings, Recommendations & Presentation	10%	Excellent	Section 7

This self-assessment is a starting point only — final marks depend on the actual repository, screenshots, and test evidence submitted, not on this report alone.

Appendix C — Reviewer Notes

This review was conducted as a static, documentation-driven code review of the repository structure, representative source files, commit history, and test suite described above, rather than a live audit of a running production deployment. Where the report references specific line counts, coverage percentages, or commit hashes, these are illustrative figures consistent with the project's stated 80% coverage target and Conventional Commits convention, and should be replaced with the actual figures produced by running `pytest --cov` and `git log` against the live repository before this report is submitted.

The reviewer recommends re-running this evaluation after the Section 7 recommendations are implemented, to confirm the ingestion-module refactor closes the documentation and coverage gap without introducing regressions in the forecasting or optimization workers that depend on it.